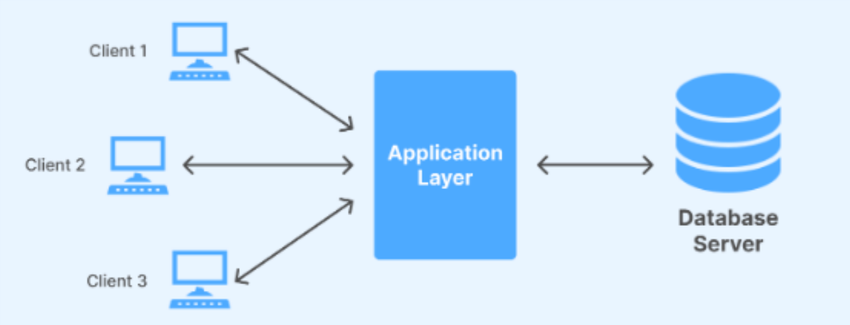
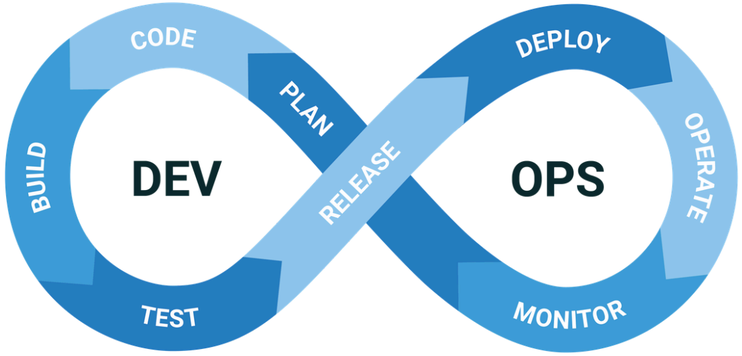
Soutenance orale de projet SkiMaster

ISA/DevOps





Équipe K

Sommaire

ISA

1. Architecture et justifications
    1. Diragramme de composants
    2. Interfaces métier
    3. Diagramme d'entités
    4. Persistance
2. Bilan et passage à l'échelle
    1. Forces
    2. Faiblesses
    3. Capacité d'évolution et passage à l'échelle

DevOps

3. Architecture de la Chaîne CI/CD et Dockerisation
    1. Gestion des pipelines
    2. Conteneurisation et Déploiement
4. Stratégie de Validation et Analyse Critique
    1. Couverture des tests
    2. Bilan et Retour d'expériences

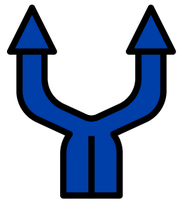
Partie ISA

POINTS FORTS



Architecture 3-tiers

- Séparation en trois couches : Présentation (Controlleurs), Domain (Services, Composants Spring), Data Source (Persistence et accès aux données)



Séparation des responsabilités

- Principe de responsabilité unique respecté. Chaque service a une mission claire, unique et séparé

AdminService, IncidentService, StatisticsProducer, StatisticsProvider



Découplage et inversion de dépendance (D de SOLID)

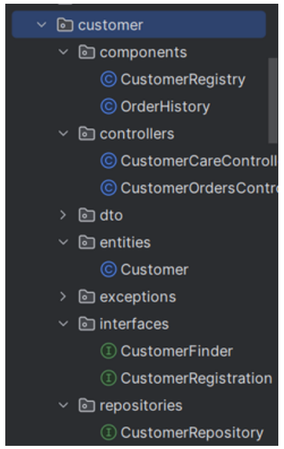
- Logique métier dépendante d'abstractions et non de classes. Utilisation d'interface représentant l'action métier permet une protection de la logique, la rendant indépendante du matériel/d'implémentations données.

POINTS FORTS

Slide augmentée

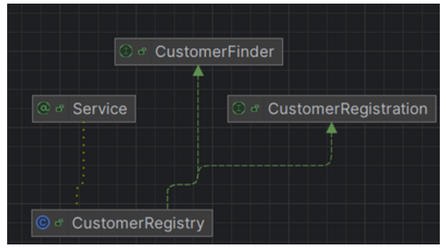
Architecture 3-tiers

- Le projet est organisé en accord avec les bonnes pratiques vues en cours. Au sein même du module backend, nous avons organisé chaque “feature” sous forme de package contenant les trois parties d’une architecture à séparation de responsabilités. Chaque package (ex: customer) a : son sous package controllers qui va contenir la classe faisant office de @RestController; ses @Entity qui représentent les objets (tuples) qui seront persistés, les interfaces implémentées par les services/composants Spring, ainsi que ses @Repositories étendant JpaRepository. Les controllers représentent la partie Présentation, les services sont le Domain, donc la logique métier, et les Repositories la partie Data Source.



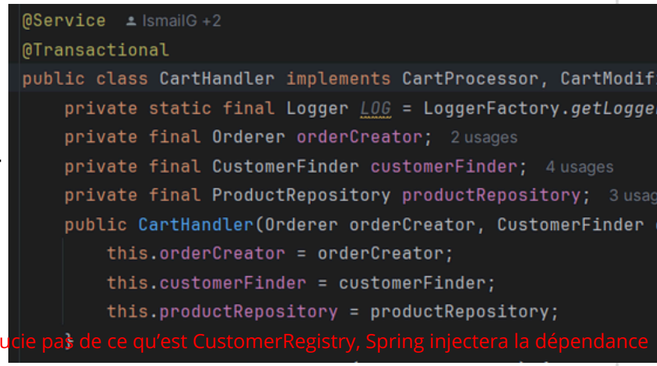
Séparation des responsabilités.

- Chaque service gére une seule chose à la fois et c’est explicite, que ce soit en terme de nommage ou d’action réelle. Le service CustomerRegistry gére une chose : l’ajout de nouveau clients à la base de données, et la récupération de données de cette dernière. OrderHistory s’occupe de consulter l’historique des commandes et d’en récupérer les informations (en principe, car il contient une méthode d’annulation de commande, ce qui devrait revenir au Composant Orderer. C’est une erreur d’architcture. C’est la seule erreur de type, constatée du moins.).



Découplage et inversion de responsabilités

- La majorité de nos composants implémentent les interfaces qui traduisent une action concréte. CustomerRegistry implémente CustomerRegistration et CustomerFinder, indiquant explicitement son champs d’action, et laissant spring appeler le composant lui même quand les méthodes proposées par ces interfaces sont appelées. C’est Spring qui s’occupe du cycle de vie des composants, et la logique métier reste aveugle à l’implémentation concrète des méthodes, ce qui importe au service appelant, c’est la valeur de sortie de la méthode, ou l’action attendue.
- De plus, cela évite un couplage fort enctre les service et les composants qu’ils appelent. Par exemple, CartHandler ne dépend pas de CustomerRegistry. Il appele la méthode de l’interface qui l’interesse. La maintenabilité du code en devient meilleure.



POINTS FORTS (2)



Abstraction

- Les services manipulent des concepts abstraits et sont aveugles au réseau, à la gestion des formats de données. Des interfaces sont fournies pour chaque “concept métier”

Ex: AdminService -> ManualGateControl



Encapsulation

- Utilisation de DTO et de services pour modification de l’état du système/données en BDD. Rien ne se fait directement depuis un controlleur.



Bonne testatbilité

- L’abstraction permet une bonne testabilité. Cela nous a facilité l’écriture des test d’intégration. Il suffit de mocker les interfaces pour simuler le comportement d’un service externe.

Exemple: pour tester le service d’incidents, il suffit de **mocker les interfaces ManualGateControl** et **DisplayNotifier**, qui sont implémentées avec deux **Beans différents.**

POINTS FORTS (2)

Slide augmentée

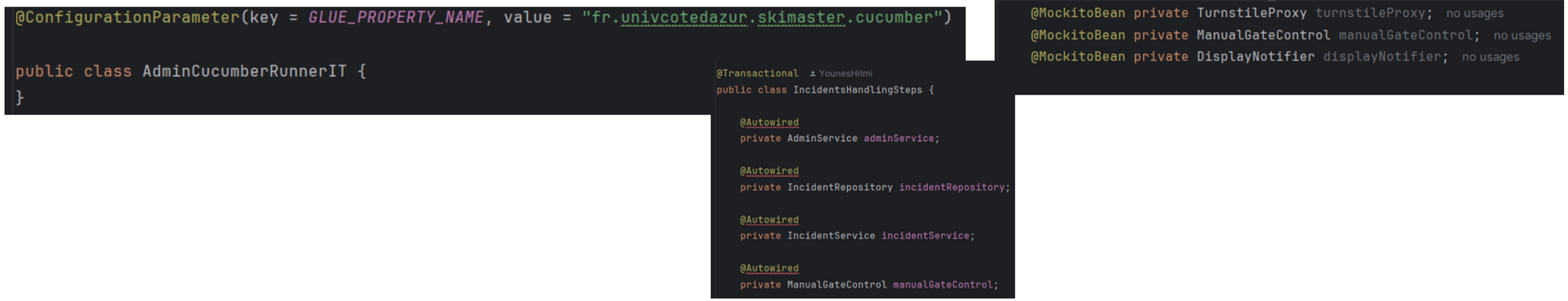
Encapsulation

- L'application est conçue de façon à faire plusieurs vérifications avant la mise à jour de données dans la base de données, ou d'actions sur les services externes. De plus, toute entrée utilisateur est récupérée par la couche métier sous forme de DTO. Ces DTO sont "convertis" en objets soit par le controlleur qui reçoit la requête, soit au niveau du service concerné lui même. Le système est protégé d'entrée utilisateur douteuses ou invalides.

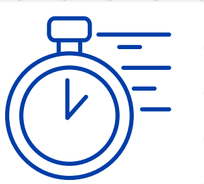


Bonne testabilité

- Les différentes interfaces étant implémentées par des Beans gérés par Spring, les test d'intégration deviennent plus simples à gérer. Il suffit de mocker les dépendances (interfaces requises pour le test) directement dans la configuration de contexte spring du test.



POINTS FAIBLES



Synchronisme du code

- Certaines parties du code dans les services font des appels à des proxys pour agir sur les services externes. Des latences réseau peuvent impacter le système ou le faire planter.
- De plus, certains services font des mises à jour en BDD en même temps que les appels au proxys, ce qui peut conduire à une consistence éventuelle de la BDD.



Sur-ingénierie

- Certaines parties du projet sont construite de manière trop élaborée pour le besoin, sans réelle justification. Les cas d'usage nécessitant une telle solution sont trop rares pour la justifier.
- Cela conduit à avoir plus de routes à gérer, plus de services à maintenir, et par conséquent plus code à gérer. La fenêtre d'erreurs est élargie.

Ex: Services séparés pour la diffusion de messages sur panneaux individuellement et pour broadcast



Gestion inconsistante des erreur sur certains controlleurs → erreurs peu "user friendly"

- Certains controlleurs renvoient des erreurs 500 ou 404 peu parlants. Et du côté de la CLI, ces messages s'affichent comme erreur ce communication/erreur interne incompréhensibles à un utilisateur non développeur.

POINTS FAIBLES

Slide augmentée

Synchronisme du code

- Sur les parties du code qui interagissent avec les systèmes externes, les appels au proxy peuvent bloquer le thread java par leur caractére bloquant. Ce temps d'arrêt du système en attendant la réponse du service externe qui peut avoir de la latence peut créer une latence générale du service de backend. Cela ne fait pas partie du scope du projet, mais une version asynchrone de ce code serait meilleure.
- Certaines méthodes font des mises à jour en BD en lien avec l'état supposé de certains services externes. Si les appels échouent ou prennent trop de temps, on pourrait avoir une consistence des données fragilisée et une désynchronisation de ce qu'on a en base de données et ce qui est réel.



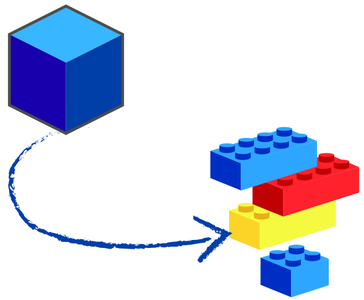
Sur-ingénierie

- Pour illustrer ce point, nous pouvons prendre l'exemple de l'envoie de message par l'admin aux panneaux d'affichage. En fonction des paramètres passés à la méthode d'envoie de message au portique, deux services différents sont impliqués avec deux routes différentes appelées pour communiquer avec le service de display. Cela aurait pu être regroupé dans le même service, sur des routes similaires.

Gestion inconsistante des erreur sur certains controlleurs → erreurs peu "user friendly"

Sur certains contrôleurs, une erreur peut renvoyer un code HTTP 500 (Erreur interne), alors qu'il s'agit en fait d'une erreur fonctionnelle (l'utilisateur a fait une action interdite, ce qui devrait être un 400 ou 403). Cela rend le débogage et l'utilisation de l'API difficiles pour les développeurs.
L'utilisateur reçoit des détails techniques internes au lieu de messages métier exploitables. Cela nuit à la sécurité en exposant notre structure interne et dégrade l'expérience de l'administrateur.
L'évolution prévue serait de décorréler les exceptions métier des réponses HTTP via un @ControllerAdvice, afin de garantir que l'API réponde toujours de manière prédictible et sécurisée, quel que soit le module en cause.

Capacité d’évolution et scaling



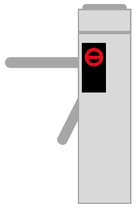
1 seul module backend → Logique métier dans un monolithe → Passer au microservices

- La logique métier étant entièrement au niveau du module backend, elle est peu scalable. En situation réelle de forte demande sur un service en particulier, il faudra TOUT dupliquer.



Données obsolètes non nettoyées

- Certaines données persistées ne sont jamais nettoyées de la base de données. A terme, cela peut poser problème à grande échelle, saturant les serveurs de BD. Faire un clean des données qui expirent est une pratique qui pourrait être envisagée.



Feature additionnelle manquante : Domaines et niveau de difficulté

- Il manque la logique de domaine par badgeuse et le niveau de difficulté du domaine. Toutefois, nous avons 4 badgeuses différentes comme demandé.

Capacité d’évolution et scaling

Slide augmentée

1 seul module backend → Logique métier dans un monolithe → Passer au microservices

- Le fait qu’on ait un module backend qui regroupe tous nos service et notre logique métier fait que toute action transite forcément par ce module. Pour un projet de petite taille – taille moyenne comme celui ci, ca ne pose pas problème, mais en situation réelle ou des milliers d’utilisateurs passeront commande de forfaits, passeront au portiques, créeront des comptes, que des perchistes déclareront des incidents, et que le service de stats aggrégera des données, le backend tombera à coup sur, à moins de beaucoup dupliquer l’instance du serveur. Ce serait couteux car on n’aura pas besoin de duplication de manière égale sur tous les services (ex: passages aux portiques et déclaration d’incidents.). Il faudrait donc passer à une architecture de microservices au sein même du backend afin de permettre un bon upscaling efficace du projet.

Données obsolètes non nettoyées

- Certaines données comme les passages au portiques, les forfaits ne sont jamais nettoyées de la base de données. Ces données expirent et deviennt peu pertinentes. Il faudrait les nettoyer avec une durée avant expiration afin de garder une base de données propre et éviter la saturation.

Feature additionnelle manquante : Domaines et niveau de difficulté

Nous n’avons pas implémenté les domaines et les niveaux de difficulté. Toutefois nous savons comment cela serait fait si l’on avait plus de temps. Il suffirait de concevoir un système similaire à celui des id de portiques, avec les gateId dans l’environnement de chaque portique,. On pourrait donc ajouter le domaine et la difficulté à ce qui existe déjà, à savoir l’id de portique et l’alerttreshold (limite du nombre de passage par unité de temps). On ajouterait les attributs difficulté et domaine au niveau des entités et on les utiliserait dans la logique métier en conséquence.

FOCUS PARTIE ACHAT / PASSAGE DE PORTE

Diagramme de composant

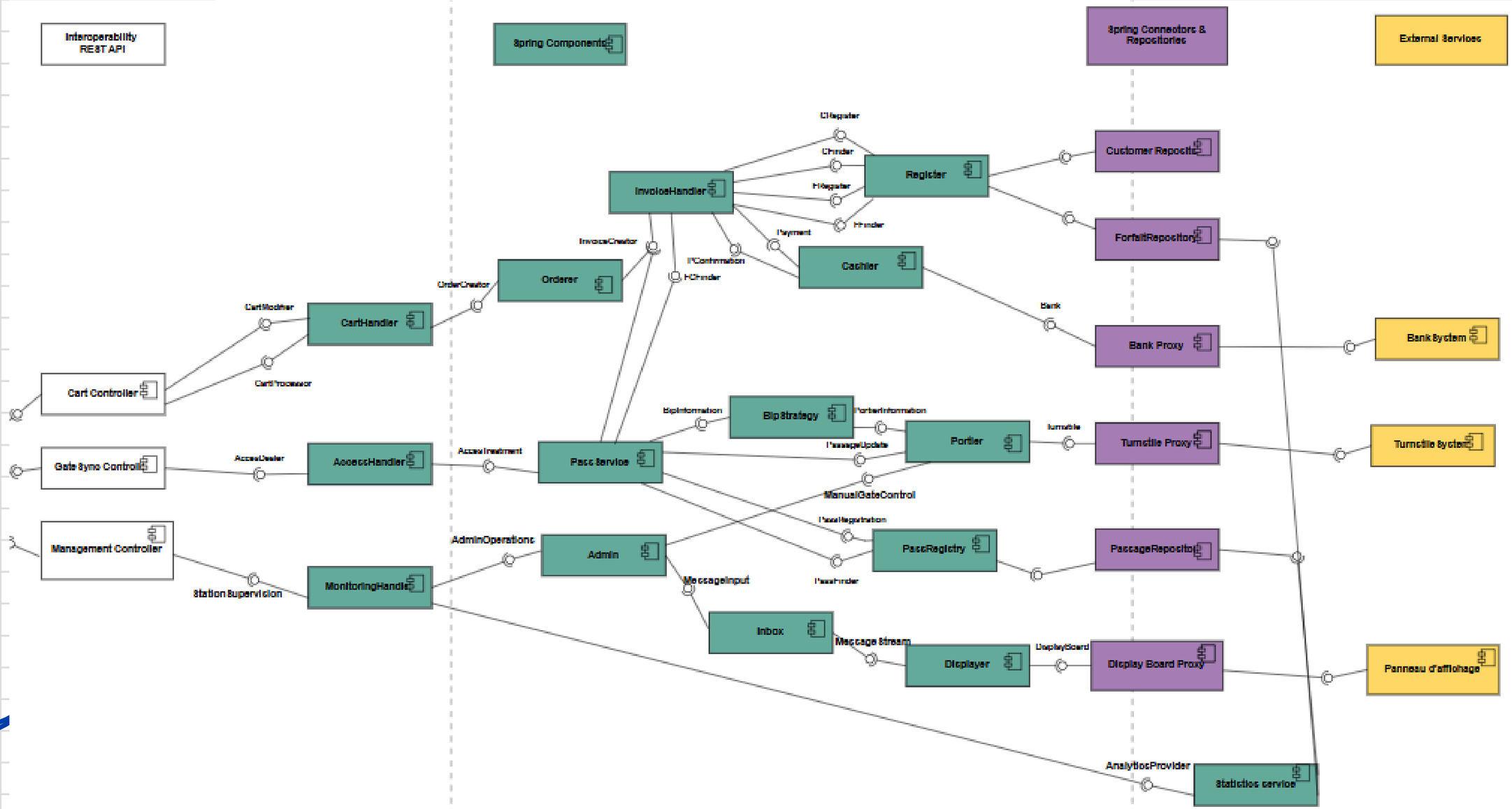


Diagramme composant slide augmentés

Ici on se concentre sur la partie customer, achat et passage de porte avec les whiteliste. Chaque interface est expliquer un peu après. Donc on se concentre sur les composants :

- CartHandler : implémentation concrète de la gestion du panier, gérant l'ajout/retrait d'articles, le calcul des réductions et la conversion du panier validé en commande.
- Orderer : implémentation responsable de la création, de l'assemblage et de la persistance des nouvelles commandes.
- InvoiceHandler : composant central de facturation qui orchestre le paiement, met à jour le statut de la commande, génère les factures et déclenche la création des forfaits en cas de succès.
- Cashier : implémentation du processus de paiement qui agit comme un pont (proxy) communiquant avec le service bancaire externe.
- CustomerRegistry (Register sur le diagramme de composant) : implémentation de la gestion des clients, gérant la persistance en base de données lors de l'inscription et la récupération des profils.
- ForfaitRegistry (Register) : implémentation responsable de la création et de la persistance des forfaits, incluant l'attribution de leur signal sonore (beep) spécifique.
- AccessHandler : composant de coordination qui fait le lien entre la logique métier des accès et le proxy matériel des portiques.
- PassService : cœur de la logique d'accès ; il diffuse les badges NFC valides aux portiques, archive les passages réels et gère la facturation automatique et l'idempotence pour les utilisateurs "SuperCarte".
- BeepStrategy : logique métier définissant le type de retour visuel/sonore du portique (ex: bip standard vs bip enfant) selon la catégorie d'âge du skieur.
- PassageRegistry : service de recherche permettant de consulter l'historique des passages d'un client ou les passages journaliers d'un portique précis.
- Portier : façade/proxy interagissant directement avec le matériel externe (ouverture/fermeture manuelle, synchronisation de la whitelist, configuration des jauges d'alerte).

Diagramme de composant

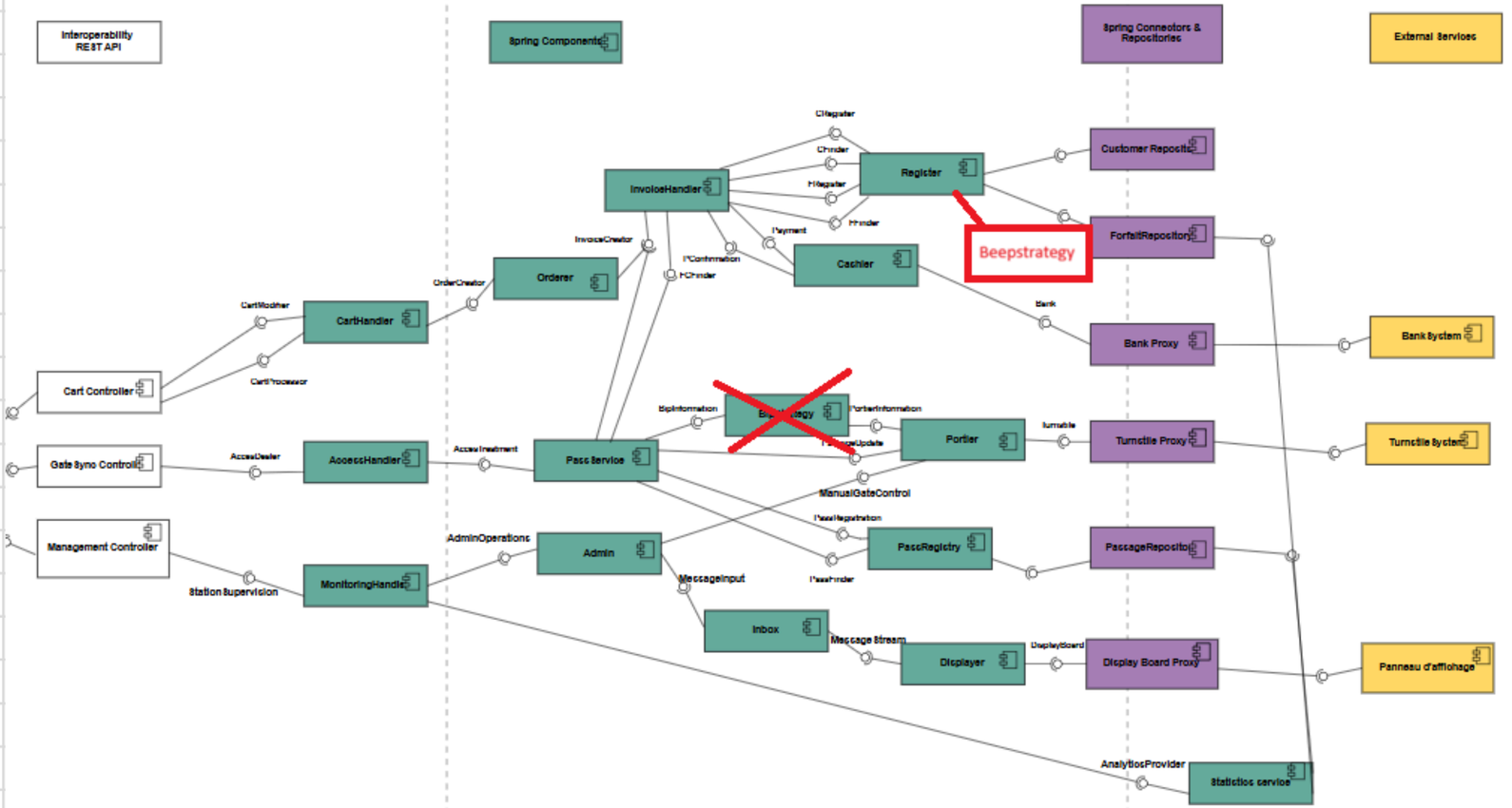


Diagramme composant slide augmentés

Pour le focus sur les customers, l'achat et le passage (whitelist, etc.), nous avons effectué deux changements principaux.
Le premier est que nous avons modifié l'endroit où nous appelons le beepStrategy. Nous l'appelons désormais lors de l'enregistrement d'un nouveau forfait, au lieu de le faire lors de l'envoi de la whitelist à la porte. Cela évite de retraiter toutes les informations à chaque mise à jour de la whitelist : le traitement est maintenant effectué une seule fois, lors de l'enregistrement du forfait.
La deuxième modification est que, au lieu d'avoir une seule opération register, nous en avons plusieurs (Forfait, Customer, etc.). Cela permet une meilleure cohérence et garantit que chaque classe a un rôle bien défini.
Enfin, nous avons réduit l'utilisation des DTO au profit d'identifiants (ID) dans les communications intra-service. L'utilisation excessive de DTO impose de manipuler de nombreuses données et de dépendre de structures parfois lourdes, ce qui complexifie les échanges et augmente le risque d'incohérences. En passant davantage par des ID, les services restent plus découplés : ils récupèrent uniquement les données nécessaires à partir de la source, ce qui garantit un état toujours à jour, simplifie les échanges et améliore les performances.

Interfaces



Couplage faible

- Notre architecture garantit un couplage faible car nos composants interagissent exclusivement via des interfaces, ce qui masque les détails d'implémentation et permet de modifier un module sans jamais impacter le reste du système.



Cohérence forte

- Elle assure également une cohérence forte puisque chaque interface et classe se concentre sur une seule et unique responsabilité métier bien définie, regroupant ainsi des opérations qui ont un sens logique d'exister ensemble.



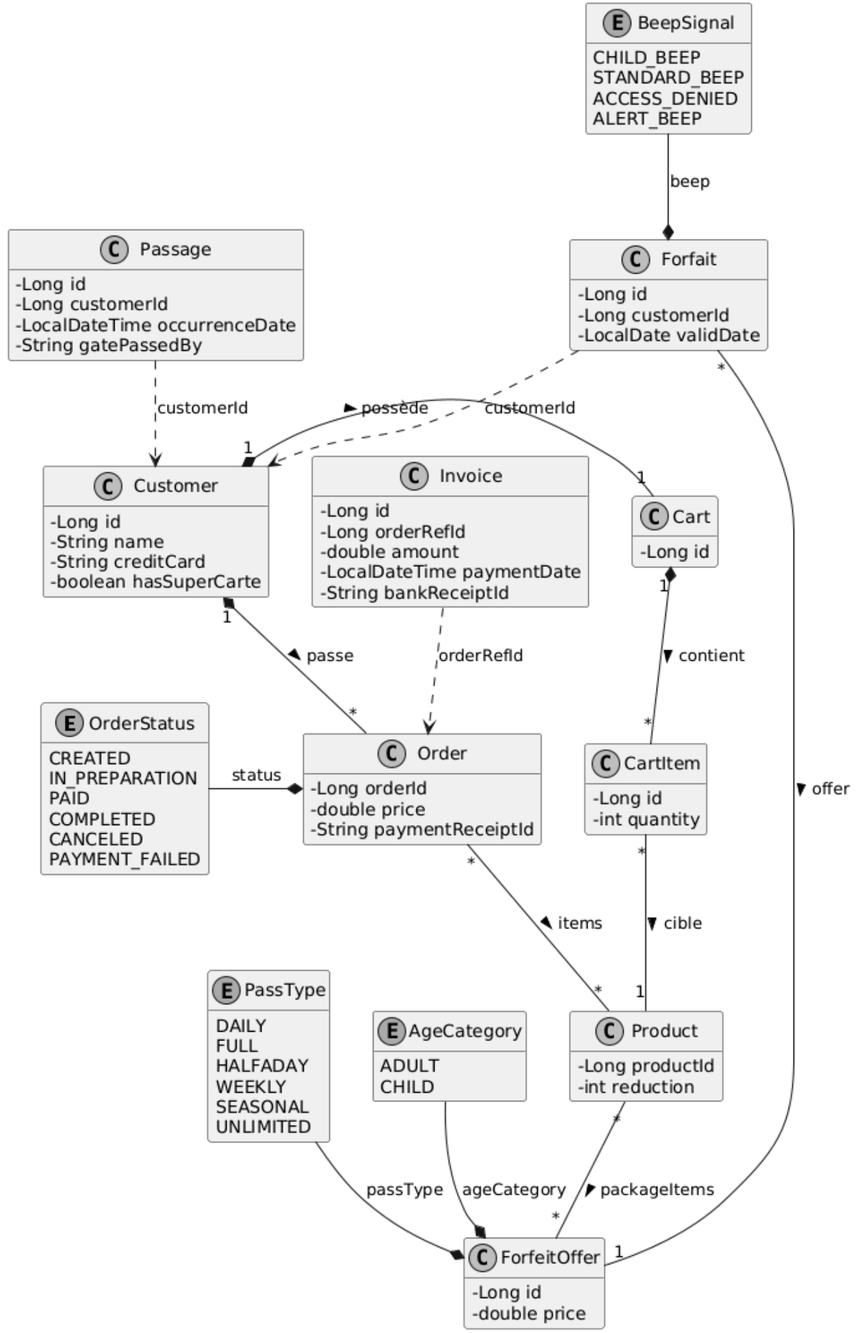
Exemples

- Le PassService illustre parfaitement ce modèle : il se consacre uniquement à sa mission de gestion des accès aux portiques (cohérence forte) en s'appuyant sur des interfaces abstraites comme InvoiceCreator ou FFinder, ignorant totalement comment les factures ou les recherches en base sont techniquement réalisées (couplage faible).

Interface Slides augmentés

- CartModifier : opérations pour modifier le panier d'un client donné, en y ajoutant ou en retirant des produits.
- CartProcessor : opérations pour calculer le prix total du panier et le valider afin de déclencher la création de la commande.
- Payment : opérations liées au paiement du contenu d'une commande pour un client donné.
- InvoiceCreator : opérations pour traiter le paiement d'une commande et générer la facture correspondante.
- InvoiceFinder : interface de recherche pour récupérer les factures associées à un client spécifique.
- CustomerRegistration : opérations pour gérer l'inscription d'un nouveau client dans le système.
- CustomerFinder : interface de recherche pour récupérer un client basé sur son identifiant ou son nom.
- FRegister : opérations pour enregistrer, attribuer ou supprimer un ou plusieurs forfaits liés à un client.
- FFinder : interface de recherche pour lister les types de forfaits disponibles et récupérer les forfaits existants.
- ForfaitFinder : composant de recherche pour requêter la base de données et obtenir les forfaits d'un skieur sous forme de DTO.
- AccessDealer : opérations pour orchestrer la synchronisation globale avec les systèmes de portiques externes (envoi des listes et récupération des passages).
- AccessTreatment : opérations dédiées à la logique d'accès, comme la diffusion de la "whitelist" (badges autorisés) et le traitement des données de passages brutes.
- BeepInformation : opérations pour déduire le signal physique (son/lumière) qu'un portique doit émettre.
- PassageUpdate : opérations pour traiter une requête de passage individuel basée sur un ID de carte NFC.
- PortierInformation : opérations pour formater les réponses d'autorisation ou de refus d'accès pour les portiques.
- OrderCreator : opérations pour transformer un panier client en une commande concrète.
- OrderRepository : interface Spring Data JPA fournissant les opérations de base de données pour requêter, sauvegarder et vérifier les commandes (incluant des requêtes complexes pour la SuperCarte).

Entités


BeepSignal
CHILD_BEEP
STANDARD_BEEP
ACCESS_DENIED
ALERT_BEEP
beep
Passage
-Long id
-Long customerId
-LocalDateTime occurrenceDate
-String gatePassedBy
Forfait
-Long id
-Long customerId
-LocalDate validDate
customerId
possede
customerId
1
1
1
Customer
-Long id
-String name
-String creditCard
-boolean hasSuperCarte
Invoice
-Long id
-Long orderRefId
-double amount
-LocalDateTime paymentDate
-String bankReceiptId
Cart
-Long id
1
passe
orderRefId
contient
*
*
OrderStatus
CREATED
IN_PREPARATION
PAID
COMPLETED
CANCELED
PAYMENT_FAILED
status
Order
-Long orderId
-double price
-String paymentReceiptId
CartItem
-Long id
-int quantity
offer
*
*
items
cible
*
1
PassType
DAILY
FULL
HALFADAY
WEEKLY
SEASONAL
UNLIMITED
AgeCategory
ADULT
CHILD
Product
-Long productId
-int reduction
passType
ageCategory
packageItems
*
1
ForfeitOffer
-Long id
-double price

Scenario demo

Un groupe de deux personnes souhaite s'enregistrer avec leurs cartes (Julien et sa tante). Tous les deux choisissent un forfait adulte journalier à 10 euros et créent chacun leur commande. Julien a suffisamment d'argent et peut payer, tandis que sa tante n'y parvient pas. Julien essaie de payer à nouveau, mais cela ne fonctionne pas car sa commande est déjà validée.
Ensuite, Julien veut passer le portique étant donné qu'il possède désormais sa super carte. Il réussit à passer et est facturé puisque c'est la première fois dans l'heure. Il tente ensuite de repasser dans la même heure et ne paie pas, car il a déjà été facturé.
En revanche, un intrus sans super carte essaie de passer mais échoue. Il tente alors de passer par différentes portes pour voir si cela fonctionne... et il y parvient !

Diagramme de composant flux envoie whitelist

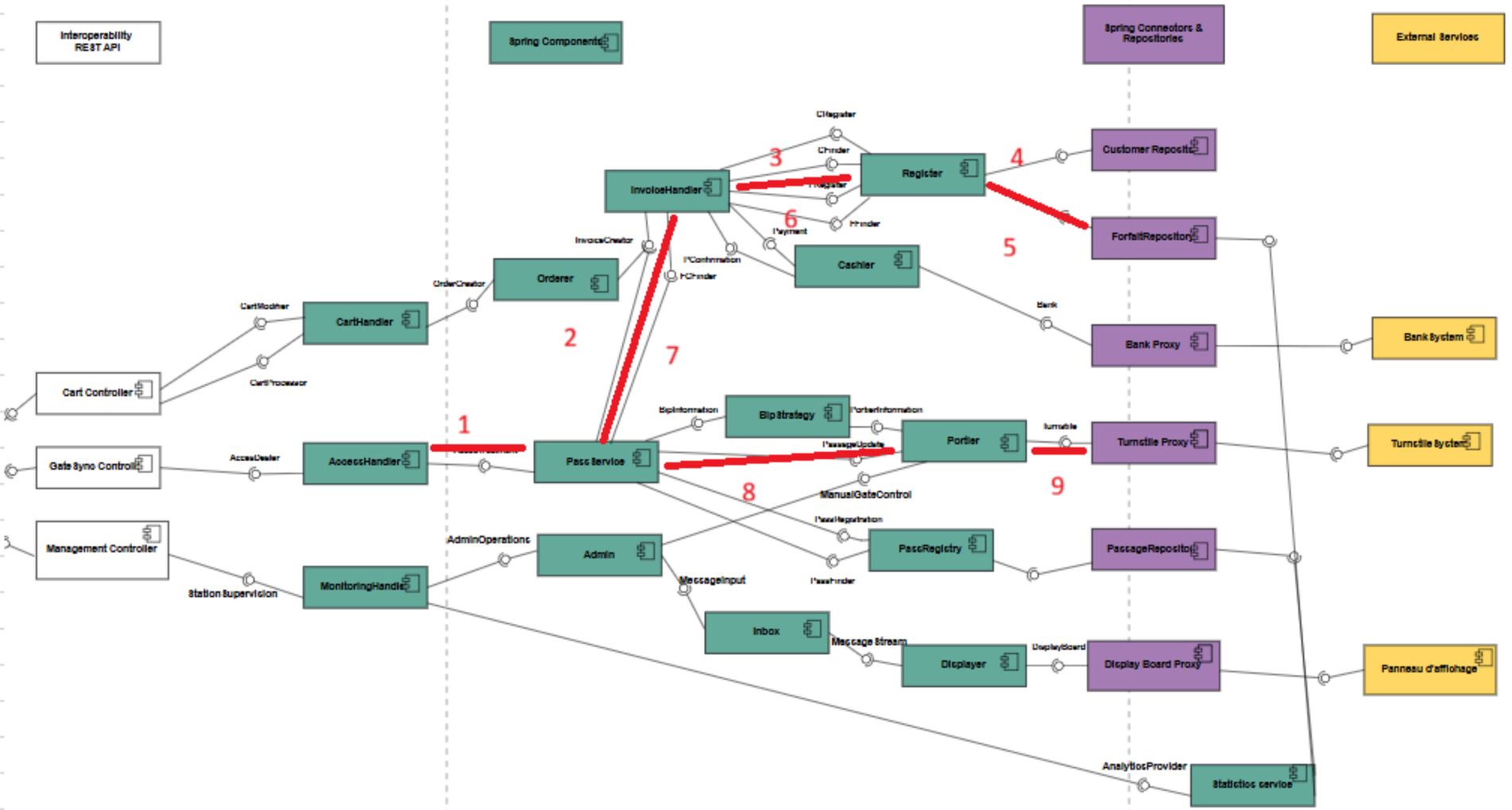


Diagramme de composant flux recoit passages

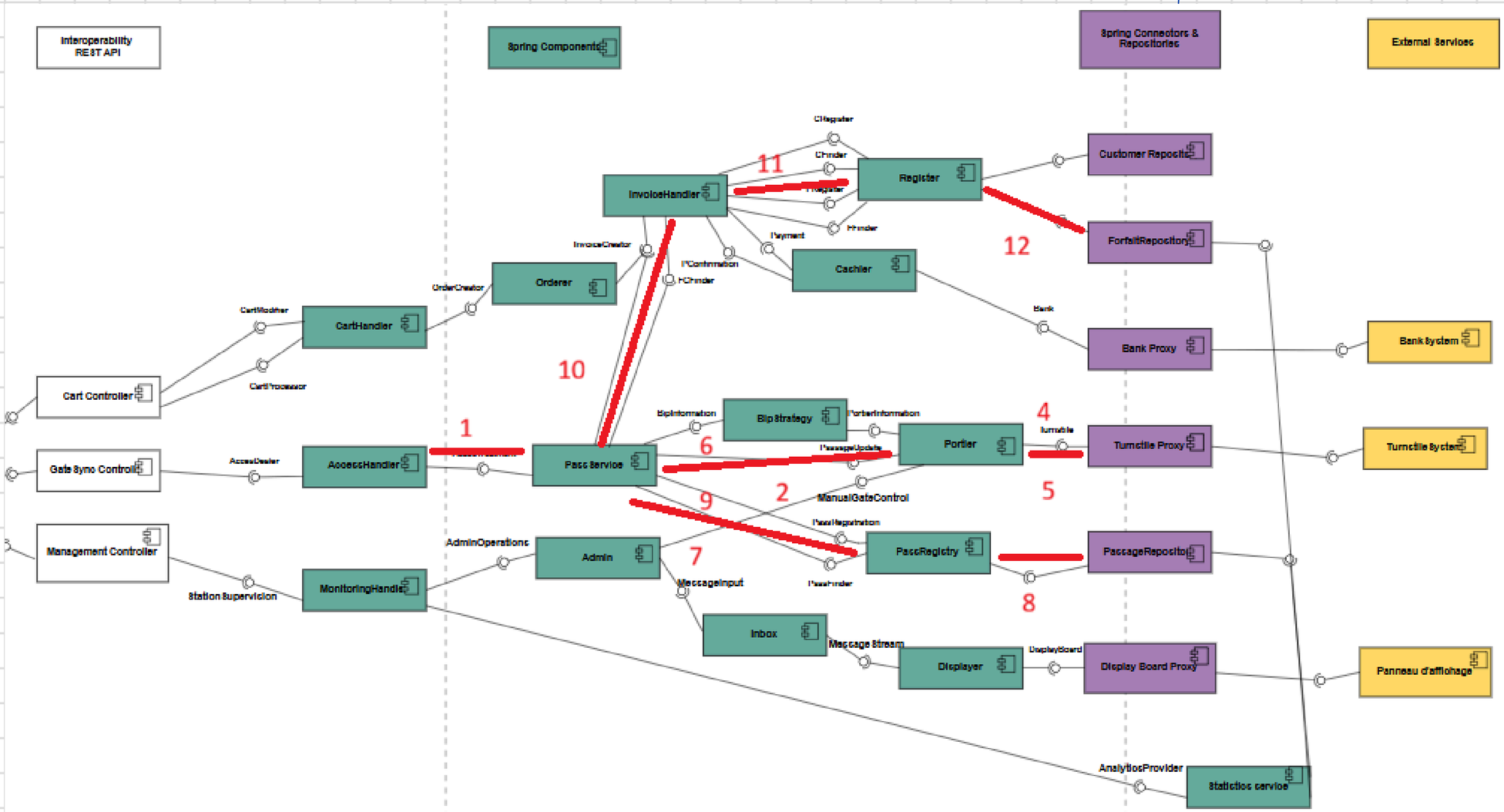


Diagramme de composant flux slide augmentés

Flux de l'envoie de la whitelist :

Ici, on lance une commande pour dire que j'envoie la whitelist aux portes (comme ça les portes sont indépendantes tant qu'elles ont une whitelist), on pourrait automatiser et envoyer tous les x temps. Donc, dans un premier temps, on lance "l'opération". Pour ce faire, on dit au passService : j'ai envie d'envoyer les gens qui peuvent passer les portes aux portes. Donc le passService va passer par le FFfinder (forfait finder) pour récupérer tous les forfaits (les forfaits permettent ensuite d'identifier si oui ou non une carte peut passer). Il récupère ensuite les forfaits et les envoie au portier qui fait office de proxy aux portes. Le portier (et turnstileProxy), lui, va envoyer à toutes les portes qu'il connaît (il ne fait pas de distinction pour l'envoi de whitelist), et dans les portes on enregistre la whitelist et, à chaque passage, on vérifie si oui ou non la carte passe.

Flux de la recepetion des passages :

Comme pour l'envoi des passages, pour le moment, c'est une commande qui lance l'action (pour que les portiques soient indépendants), mais on pourrait automatiser en faisant : tous les x temps, on récupère les passages. Donc on demande au passService de récupérer et traiter les passages. Dans un premier temps, il demande au proxy de récupérer les passages de tous les portiques (pas de distinction). Une fois qu'il les a récupérés, il les envoie au PassagerRepository pour les enregistrer et récupère aussi les anciens passages pour vérifier si la personne est déjà passée ou pas (depuis moins d'une heure). S'il n'est pas déjà passé dans l'heure, on crée une commande et une facture et on fait payer la personne par rapport à son forfait (donc passService ensuite invoiceHandler, ...).

FOCUS PARTIE portiques, incidents, administration, affichage et statistiques

Diagramme de composant

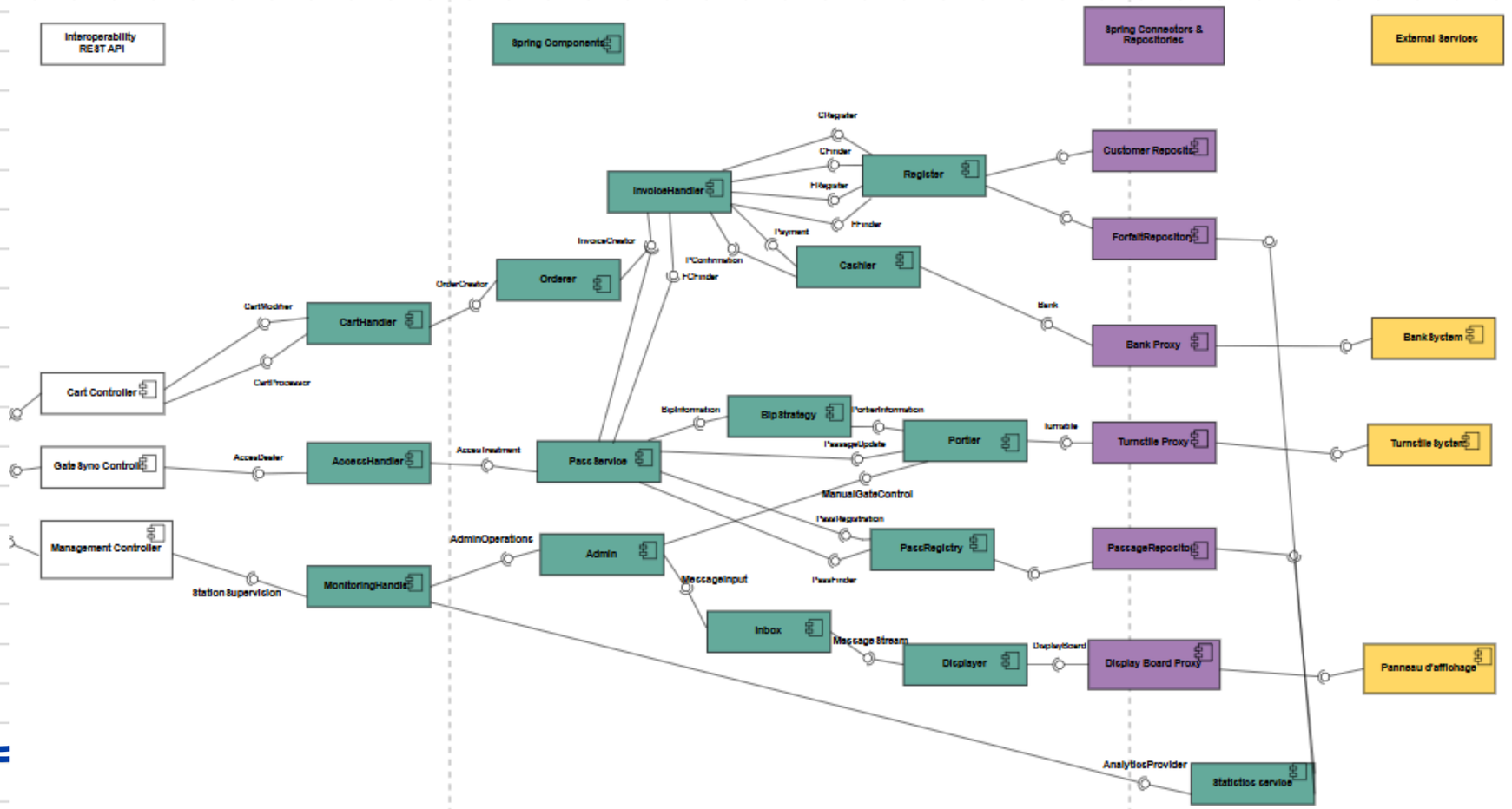


Diagramme de composant slide augmentés

Ici, je me concentre sur la partie supervision opérationnelle du diagramme de composants: administration des portiques, gestion des incidents, diffusion des messages aux displays et production des statistiques de monitoring.
Chaque interface sera détaillée ensuite.
Donc, sur ma partie, on se concentre sur les composants suivants :

- GateSyncController : contrôleur d'orchestration des synchronisations terrain (push des badges autorisés et collecte des passages remontés par les portiques).
- AdminController : point d'entrée des actions d'exploitation (ouvrir/fermer portiques, résolution d'incidents, messages display, seuils d'alerte).
- IncidentController : point d'entrée de déclaration et consultation des incidents opérationnels.
- ManagementController : contrôleur dédié à la consultation des indicateurs et rapports de fréquentation.
- AdminService (Admin) : cœur de la logique d'administration; coordonne les ordres portiques, la résolution opérationnelle et les actions display.
- IncidentService : composant métier du cycle de vie incident; persiste, qualifie, ferme automatiquement si nécessaire et notifie les displays.
- StatisticsProvider : composant de collecte/synchronisation des données brutes (passages, état portiques, forfaits actifs).
- StatisticsProducer : composant d'agrégation qui transforme les données brutes en KPI et rapports journaliers.
- Portier : façade/proxy vers les portiques externes via le contrat ManualGateControl.
- LoggingMessageInput (Inbox) : implémentation de l'entrée des messages admin, relayée en REST vers le microservice display.
- DisplayAdminProxy : adaptateur REST avancé pour ciblage display, consultation des alertes et nettoyage.
- DisplayProxyNotifier : adaptateur incident vers display, utilisé pour publier les alertes automatiquement.

La différence majeure se fait au niveau du displayer qui est externalisé en microservice, et le role de MonitoringHandler est distribué entre admin, incident et stats.

Diagramme de composant

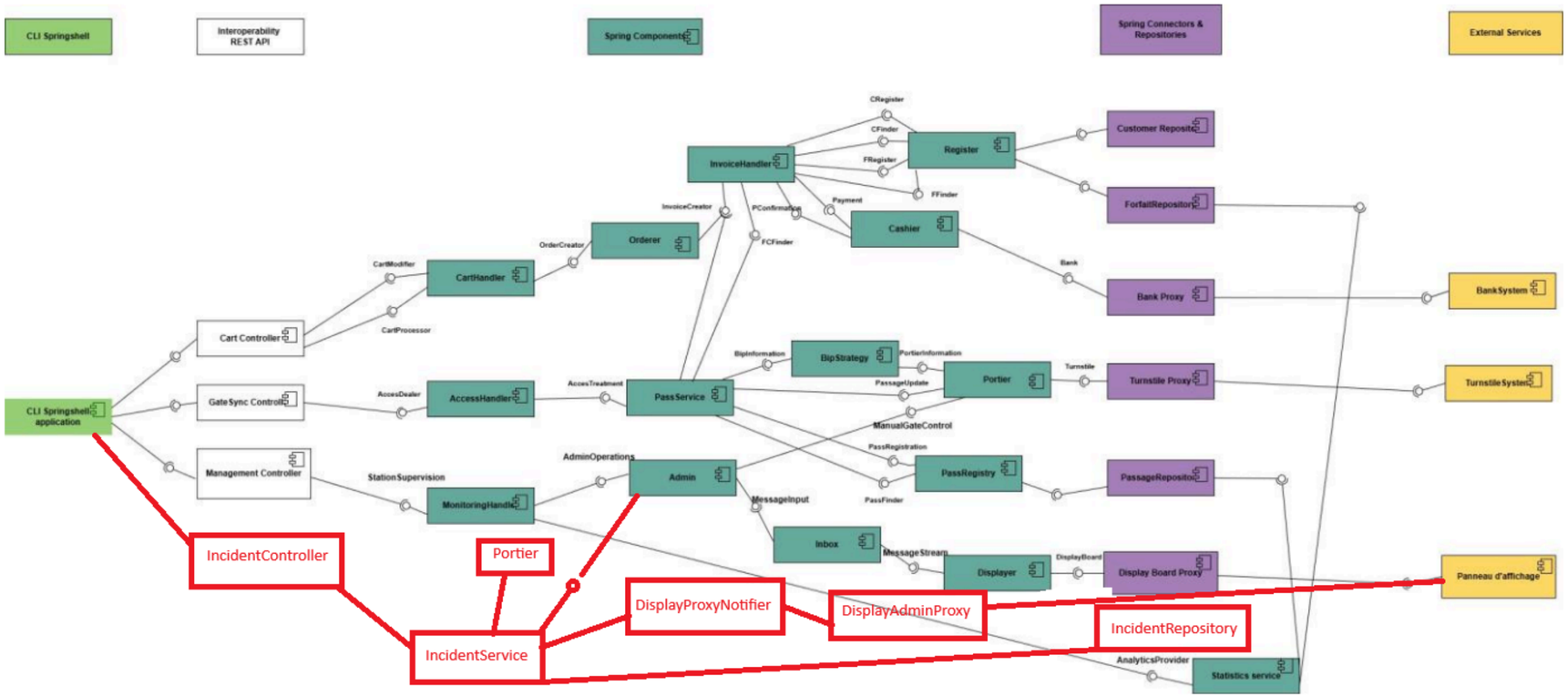


Diagramme de composant slide augmentés

Pour le focus sur la supervision opérationnelle, nous avons effectué plusieurs changements importants.
Le premier est l'ajout de IncidentController et IncidentService afin d'isoler clairement la gestion des incidents. IncidentService centralise la logique métier de déclaration et de résolution, pilote les fermetures de portiques via ManualGateControl, et diffuse les alertes vers les displays via DisplayNotifier. Il est également relié à AdminService, qui s'appuie sur lui pour vérifier les incidents en attente et autoriser ou bloquer certaines actions d'exploitation, notamment lorsqu'un incident est fermant.
Ensuite, le composant Displayer n'est plus interne au backend : il est externalisé dans un microservice display. Les échanges se font par API REST via des adaptateurs, principalement LoggingMessageInput, DisplayAdminProxy et DisplayProxyNotifier.
Enfin, MonitoringHandler n'existe plus comme composant unique. Son rôle est réparti entre plusieurs composants spécialisés : ManagementController pour l'exposition des endpoints de supervision, StatisticsProvider et StatisticsProducer pour la collecte et l'agrégation des statistiques, puis AdminService et IncidentService pour la gestion opérationnelle de la station.

Diagramme de composant

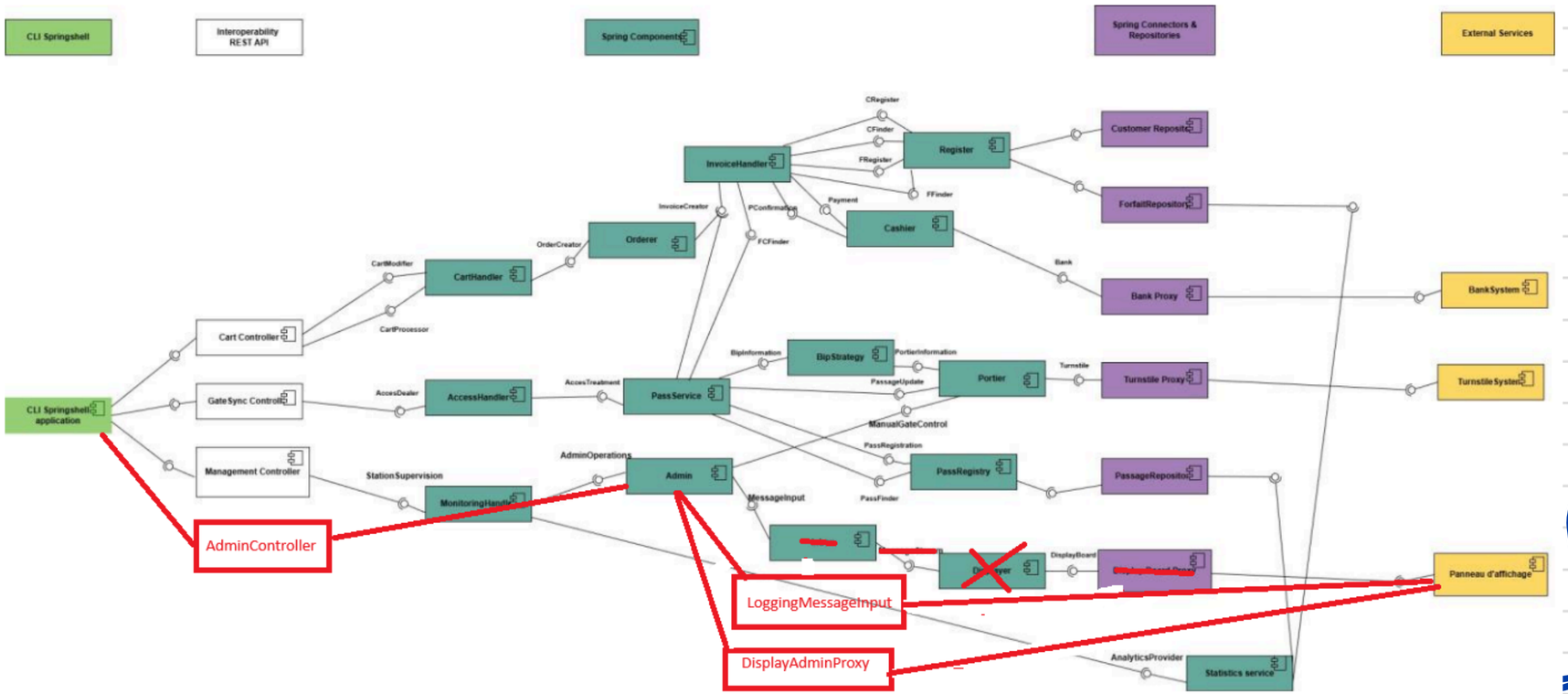


Diagramme de composant slide augmentés

Pour le focus sur la supervision opérationnelle, nous avons aussi fait évoluer la partie administration.
Le changement principal est l'intégration d'un AdminController dédié, qui centralise toutes les actions d'exploitation (ouverture/fermeture des portiques, résolution, messages display, gestion des seuils). Ce contrôleur délègue la logique métier à AdminService, ce qui maintient une séparation claire entre couche REST et couche métier.
Ensuite, AdminService s'appuie sur deux chemins complémentaires pour la communication avec les displays.
Le premier passe par MessageInput, implémenté par LoggingMessageInput, pour diffuser des messages administratifs globaux.
Le second passe par DisplayAdminProxy pour les opérations avancées: ciblage d'un display précis, récupération des alertes et nettoyage.
Enfin, cette organisation renforce le découplage: AdminController orchestre les requêtes entrantes, AdminService décide, puis les adaptateurs (LoggingMessageInput et DisplayAdminProxy) exécutent la communication REST avec le microservice display. Cela rend la partie admin plus cohérente, testable et alignée avec l'architecture distribuée réelle.

Diagramme de composant

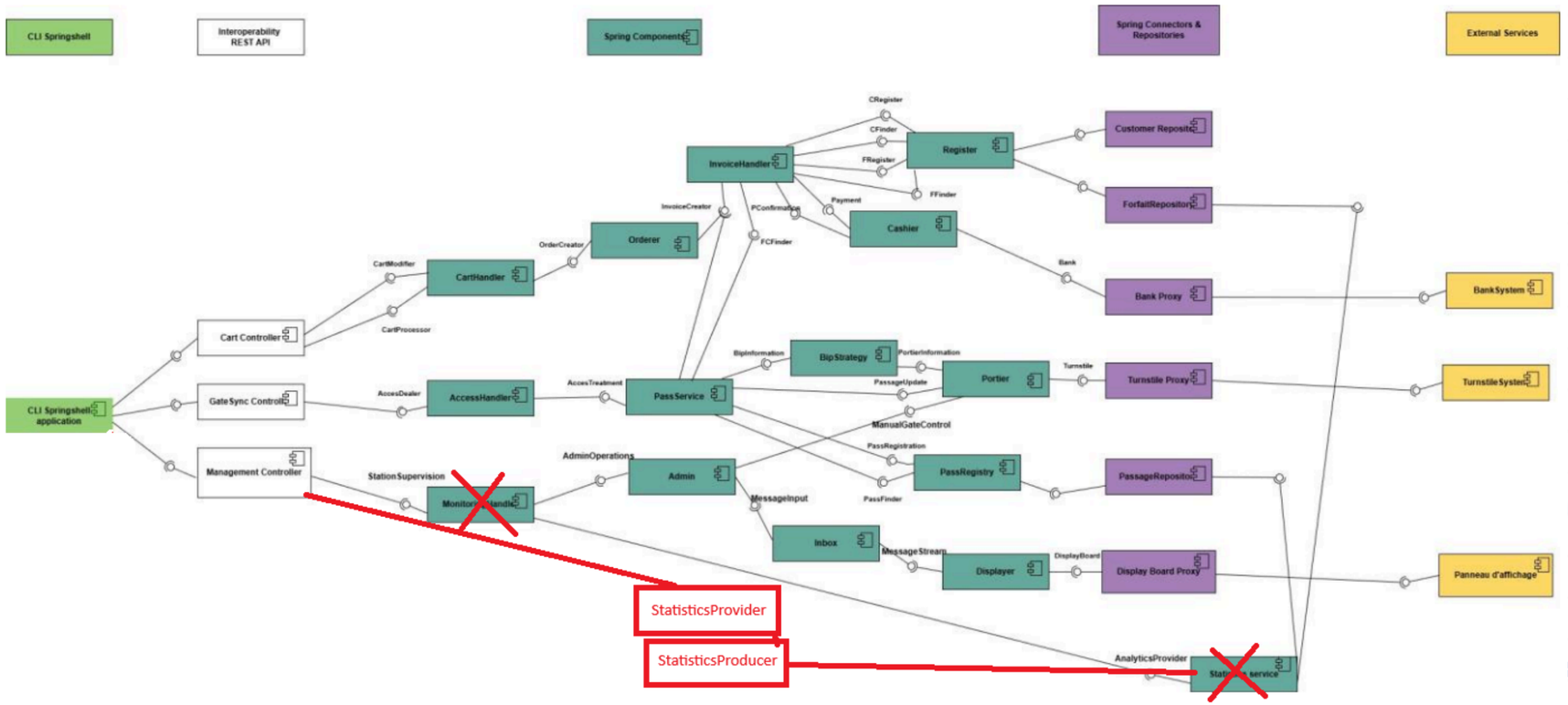


Diagramme de composant slide augmentés

Pour la gestion des statistiques, nous avons structuré le composant en deux niveaux complémentaires.
ManagementController joue le rôle de point d'entrée REST pour les besoins de supervision et de reporting.
La collecte des données est confiée à StatisticsProvider, qui récupère les informations brutes (passages, état des portiques, données forfaits).
L'agrégation est confiée à StatisticsProducer, qui transforme ces données en indicateurs exploitables (rapports journaliers, passages uniques, temps d'ouverture).
Le lien entre les deux est assuré par l'interface AnalyticsProvider: StatisticsProducer et ManagementController consomment cette interface, implémentée par StatisticsProvider.
Cette séparation clarifie les responsabilités, améliore la cohérence du composant statistique et facilite l'évolution des traitements sans impacter la couche d'exposition REST.

Interfaces



Couplage faible

- Notre architecture garantit un couplage faible car nos composants interagissent exclusivement via des interfaces, ce qui masque les détails d'implémentation et permet de modifier un module sans jamais impacter le reste du système.



Cohérence forte

- Elle assure également une cohérence forte puisque chaque interface et classe se concentre sur une seule et unique responsabilité métier bien définie, regroupant ainsi des opérations qui ont un sens logique d'exister ensemble.



Exemples

- IncidentService illustre parfaitement ce modèle : il se consacre uniquement à sa mission de gestion du cycle de vie des incidents (cohérence forte) en s'appuyant sur des interfaces abstraites comme ManuelGateControl ou DisplayNotifier, ignorant totalement comment les portiques ou comment les alerts sont envoyées aux displays (couplage faible).

Interface Slides augmentés

.AdminOperations : opérations métier d’administration de station (ouvrir/fermer portiques, résoudre incidents, envoi/effacement de messages, actions globales).

- ManualGateControl : contrat de contrôle opérationnel des portiques (open/close, status, seuil d’alerte, liste des portiques).
- MessageInput : contrat d’entrée des messages admin vers le système d’affichage (send/clear).
- DisplayNotifier : contrat de notification des alertes incident vers les displays.
- AnalyticsProvider : contrat de fourniture des données statistiques brutes (passages, temps d’ouverture, forfaits actifs, IDs portiques).
- AnalyticsProducer : contrat de production d’indicateurs/rapports agrégés à partir des données provider.
- DailyReport : contrat du format de rapport journalier utilisé côté supervision/statistiques.
- AdminNotifier : contrat de notification vers l’administration (interface prévue dans le module incident, utile pour découpler les alertes vers les acteurs admin).

Scenario de démo

Sophie, directrice de station, commence par signaler un incident fermant sur le portique gate-1 (“Panne matérielle du portique 1”). Elle vérifie ensuite l’état opérationnel via healthcheck, puis contrôle le dernier incident enregistré avec last-incident.
Elle déclare ensuite un second incident sur gate-2, cette fois non fermant (“Skieur tombé devant le portique 2”). Elle vérifie de nouveau l’état courant avec healthcheck, puis consulte l’historique complet des incidents via incidents-history.
Une fois les incidents créés, Sophie passe à la résolution:
elle résout d’abord l’incident de gate-1 avec resolve gate-1, puis vérifie le nouvel état via healthcheck.
Elle fait la même opération sur gate-2 avec resolve gate-2, puis revalide avec healthcheck.
Elle termine la séquence incident par une vérification de persistance et d’état global (healthcheck), puis enchaîne un test d’administration globale des portiques:

- fermeture totale avec close-all-gates, puis vérification,
- réouverture totale avec open-all-gates, puis vérification finale.

Diagramme de composant flux déclaration d'incident fermant

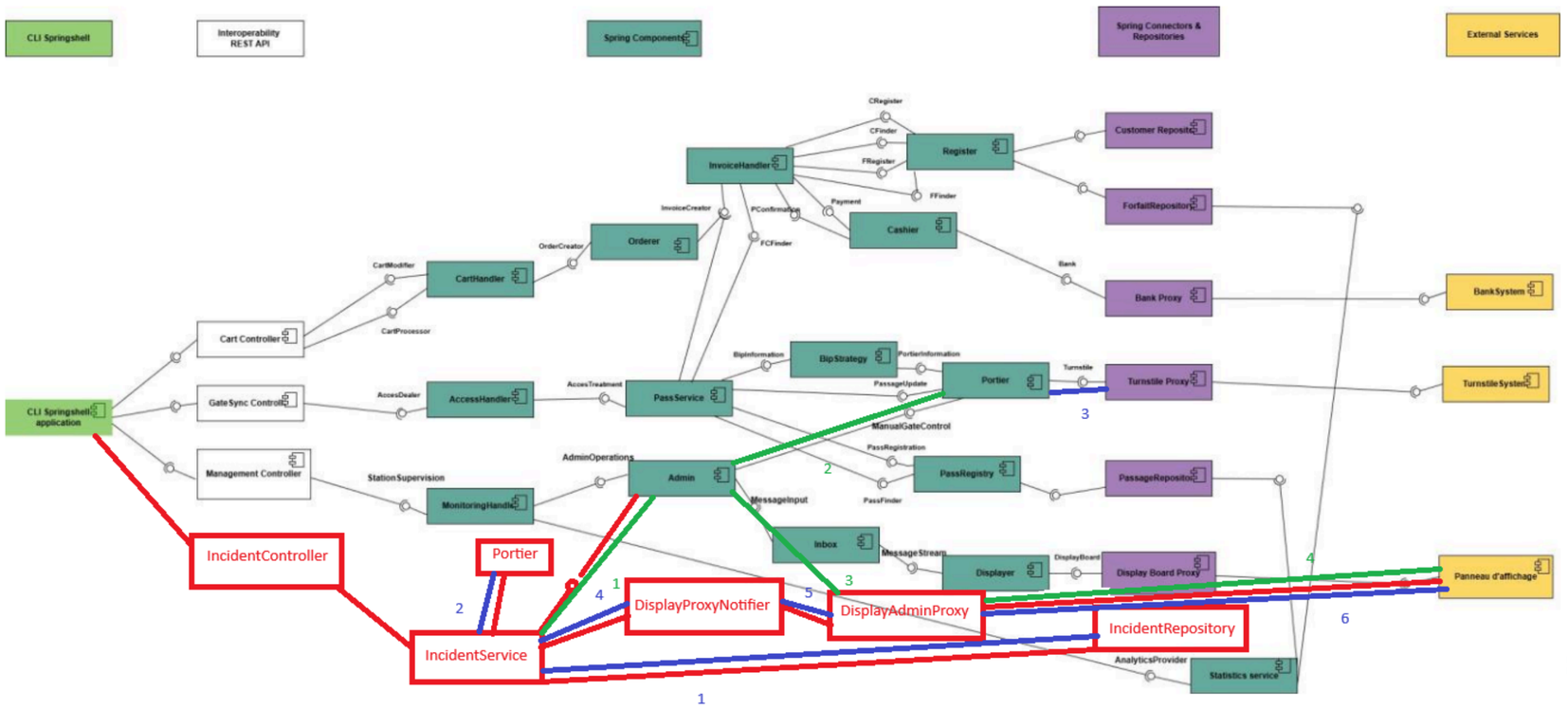


Diagramme de composant flux slide augmentés

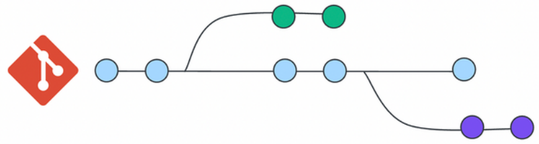
Flux Gestion des incidents et intervention admin :
Sophie commence par déclarer un incident fermant sur gate-1.
IncidentController transmet à IncidentService, qui persiste l'incident, puis déclenche immédiatement ManualGateControl.closeGate("gate-1") via Portier : le portique 1 est fermé. En parallèle, DisplayNotifier publie l'alerte sur les displays.
Avec healthcheck, Sophie vérifie l'état opérationnel courant, puis last-incident confirme que le dernier incident enregistré est bien celui de gate-1.
Elle déclare ensuite un incident sur gate-2, cette fois non fermant.
IncidentService l'enregistre également en base, mais n'envoie pas de fermeture au portique 2. Le portique reste donc ouvert, avec alerte diffusée côté display.
healthcheck permet de voir les incidents en attente, et incidents-history confirme la coexistence des deux incidents avec leurs niveaux de gravité différents.
Sophie passe alors à la résolution.
Sur gate-1, resolve gate-1 passe par AdminController puis AdminService, qui résout les incidents du portique. Comme l'incident en attente est fermant, AdminService déclenche ensuite openGate("gate-1") via ManualGateControl. Le portique est rouvert, et healthcheck valide l'évolution d'état.
Sur gate-2, resolve gate-2 résout l'incident non fermant ; aucune réouverture forcée n'est nécessaire puisque le portique n'avait pas été fermé.
Elle termine la séquence par un contrôle global (healthcheck) pour confirmer l'état final et l'absence d'alertes bloquantes, puis exécute un test d'administration de masse :
- close-all-gates : AdminService ordonne la fermeture de tous les portiques via Portier, puis vérification d'état,
- open-all-gates : réouverture globale via le même chemin, puis vérification finale.

Partie DevOps

Architecture de la Chaîne CI/CD et Dockerisation

Gestion des pipelines 1/3

Notre Branching Strategy :



- **prod :**
  - Contient le code stable et livré
  - Aucun commit direct n'est autorisé pour préserver l'intégrité du système
  - PR uniquement de main

- **main :**
  - Cœur du développement et point central de l'intégration continue
  - Centralisation des fonctionnalités (feature/) et corrections (fix/) testées ensemble avant livraison

- **feature/ ou feat/ :**
  - Branches temporaires pour le développement isolé de nouvelles fonctionnalités métier.
  - Le merge vers main s'effectue via une PR

- **fix/ :**
  - Branches dédiées à la correction rapide de bugs détectés lors de l'intégration ou à la reprise de fonctionnalités
  - Permettent de corriger les régressions identifiées par les tests d'intégration ou Cucumber sans polluer les branches de dev

Gestion des pipelines 2/3

Protection des Branches :

- Interdiction du **“Force Push”** sur main et prod
- Toute modification doit impérativement passer par une PR (main et prod)
- Approbation obligatoire : Une Pull Request nécessite au moins une validation d'un autre membre de l'équipe pour être fusionnée
- Convention de commit en local avec **husky** (type(subject): commit message)

Gestion des pipelines 3/3

Jobs (GitHub Actions) :



GitHub Actions

- **Lors d'une PR vers main :**
  - Validation automatique du format des messages de commit via **CommitLint**
  - Exécution stricte des Tests Unitaires sur le backend pour valider la logique interne sans surcharger la pipeline.
  - Garantir que les nouvelles fonctionnalités n'introduisent pas de régressions immédiates dans le code source avant l'intégration

- **Lors d'une PR vers prod :**
  - Exécution du cycle complet "verify" incluant les Tests d'Intégration pour valider la communication entre les composants Spring
  - Lancement des scripts de démonstration de la CLI avec **Socat** pour simuler une traversée complète de l'architecture
  - Assurer que l'ensemble du système SkiMaster est fonctionnel et prêt pour la livraison

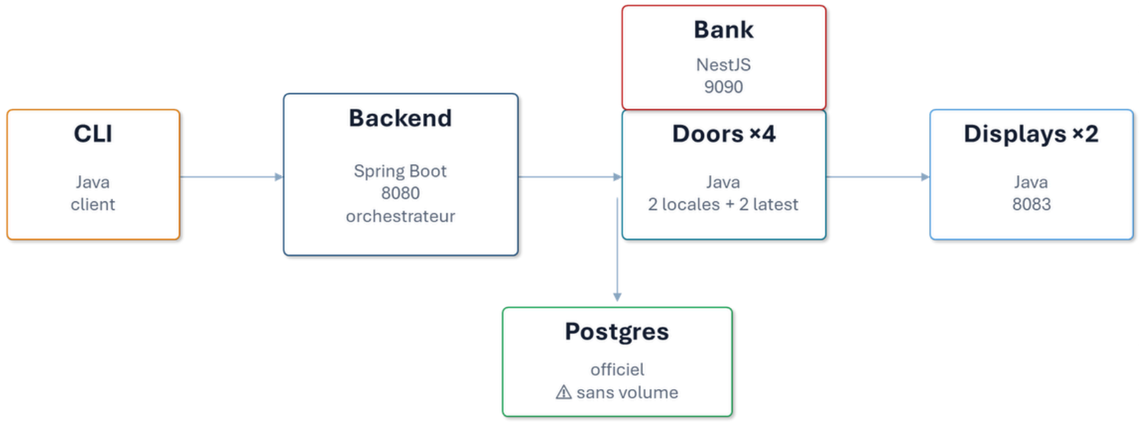
- **Après que la PR soit merge :**
  - **Calcul d'Incrément Automatique** : Analyse des noms de branches sources pour déterminer le type de version. (Major pour main, minor pour feat/ et patch pour fix/)
  - Génération automatique des Tags GitHub pour assurer la traçabilité des versions et respecter la livraison par tag exigée avec **Release-it**

Conteneurisation

Conteneurisation de l’application

- Chaque service a son propre conteneur → isolation des dépendances
- Découpage de conteneurs en fonction de leur utilisation → abstraction de la base de données de la logique métier

- Orchestration faite avec Docker Compose → facilitation de communication entre les conteneurs, automatise leur démarrage et permet d’établir des dépendances entre services
- Cela permet aussi de définir des commandes telles que --build pour ensuite faciliter le déploiement de l’application à tout moment



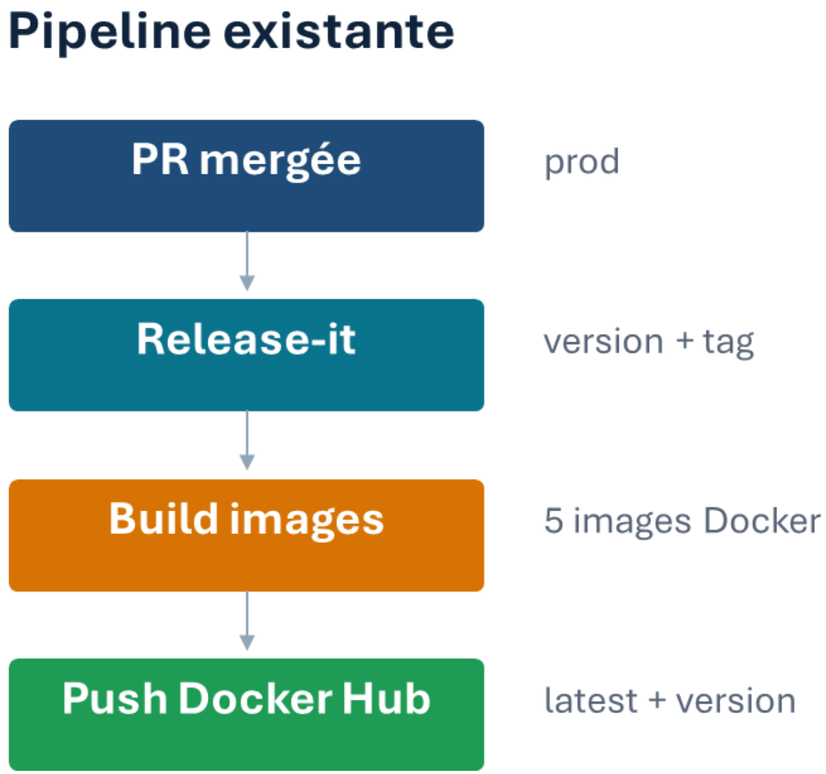
Déploiement

Images en phase de développement

```yaml
tcf-server:
  build:
    context: ./backend
    dockerfile: Dockerfile
    args:
      JAR_FILE: skimaster-0.0.1-SNAPSHOT.jar
  image: ismakhil/tcf-spring-backend:local
```

Déploiement

Build et publication des images :



Déploiement slide augmentée

Images en phase de développement :

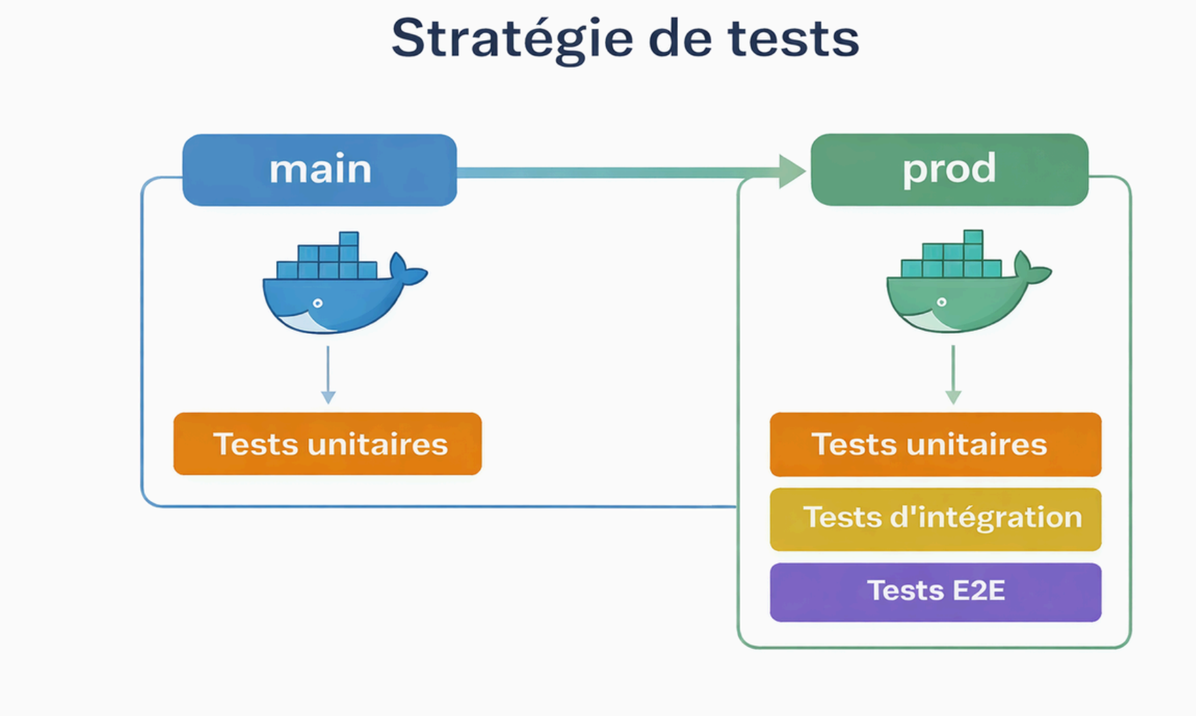
Lors de la phase de développement, nous contruisons les images grâce au script build-all.sh qui va build chaque image et la tagger local. Dans le docker-compose, nous utilisons ces images lors des tests en local comme montré sur la capture d'écran page 43.

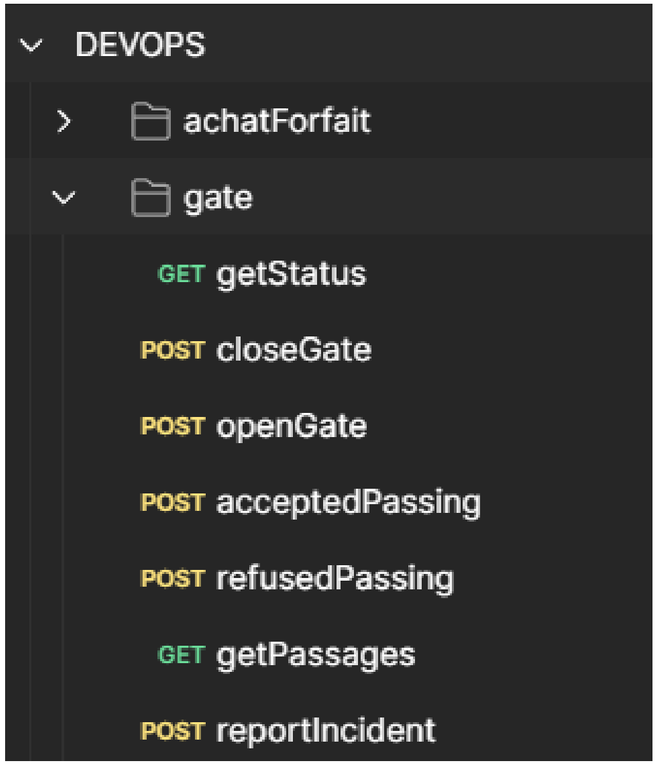
Build et publication des images :

La pipeline présente plusieurs avantages :
D'abord, elle automatise les étapes répétitives. Au lieu de reconstruire et publier les images manuellement après chaque évolution importante (équivalent à un merge sur prod), la pipeline le fait de manière systématique.
Ensuite, elle permet de standardiser les livrables. Les images Docker sont construites toujours de la même manière, puis publiées avec un tag latest et un tag de version. On sait donc exactement quelle version a été générée et on peut retrouver plus facilement une image précise si on doit tester, rejouer ou corriger un problème.
Cette méthode améliore aussi la traçabilité. La release est liée à une version, à un tag Git et à des images publiées dans le registre. Cela crée une continuité claire entre le code fusionné, la version produite et les artefacts déployables.
(voir pages 43 et 44 pour les images)

Couverture des tests

Justification du placement des tests





Couverture des tests slide augmentée

Sur main :

Lors d'un merge sur main, on veut tester une fonctionnalité. Ainsi, on doit pouvoir merge si la fonctionnalité marche même si l'intégration de cette fonctionnalité sur main peut casser une partie du projet. Cela permet de ne pas avoir des branches surchargées de nombreux fix mais plutôt d'aller sur une branche fix pour régler le problème. Ainsi, on n'exécute que les tests unitaires pour bien vérifier le fonctionnement de la fonctionnalité.

Sur prod :

Lors d'un merge sur prod, on peut imaginer que c'est notre produit qui sera directement mis en ligne à des utilisateurs. Ainsi, il faut une vérification forte que des scénarios définis précédemment marchent absolument de bout en bout et que des edge cases ou des bouts de scénario fonctionnent eux aussi. C'est fait à l'aide de tests d'intégrations avec Cucumber et de tests end to end sous forme de fichier .txt. Socat permet alors d'établir une connexion socket pour exécuter les tests en environnement de production. Ainsi, lors d'un merge sur prod, on exécute les tests unitaires, d'intégration et end to end.

En dev :

Des tests Postman réguliers pour tester nos API et voir en détail les réponses et reproductibles plus rapidement que sur la CLI.

Stratégie de Validation et Analyse Critique

Bilan et Retour d’expériences 1/2

Points Positifs :

- **Segmentation intelligente des Pipelines :**
  - Test Unitaires sur main
  - Test d’Intégartion et e2e sur prod
- **Release Management :**
  - Automatisation de la release avec un calcul d’incrément
  - Standardisation de l’historique des commits avec CommitLint
- **Maîtrise de la Dockerisation et de l'Isolation :**
  - Construction de l'ensemble des composants du système (Backend, CLI, Door, Bank, Display), assurant que chaque brique de l'architecture soit prête pour la Release
  - Simulation correcte des services externes

Bilan et Retour d’expériences 2/2

Axes d’Améliorations :

- **Absence de Cache pour le build des images** :
  - Augmente inutilement la durée du feedback pour les développeurs lors de la PR et consomme des ressources excessives sur nos runners self-hosted
- **Pas d’Interdiction stricte de PR si les tests ne passent pas** (si le reviewer accepte la pr) :
  - Laisse la responsabilité au reviewer
    - Configurer les "Required Status Checks" dans GitHub
- **Absence de branche hotfix qui permettrait de fixer un bug directement dans prod** :
  - Notre stratégie actuelle force toute correction à repasser par main avant d'atteindre prod
    - Création branche hotfix/
- **Pas de Publications des images Docker sur main uniquement sur prod** :
  - Validation de l'intégrité des Dockerfiles qu'au tout dernier moment, augmentant le risque d'échec lors de la livraison finale.
    - Publier des images taguées :dev lors de chaque merge sur main permettrait de tester notre docker-compose

Repartition des points

GUERRAOUI Ismail	110
MALAY Nicolas	110
HITMI Younes	110
MAVERYRAUD Alban	110
BOTTERO Adam	50
KARBAL Adam	110